

The Resource Divide: Data Scarcity, Compute Inequity, and Global LLM Governance

Connor Sempf and Bo Pang

School of Electrical and Computer Engineering, Georgia Institute of Technology
{csempf3, bpang42}@gatech.edu

Abstract

Large language models are positioned as universal tools, yet the resources required to train and deploy them are distributed unevenly across the world’s languages. We quantify compounding disadvantages across tokenization efficiency, memory scalability, and hardware access. Using the Llama-3 tokenizer on six languages spanning five writing systems, we find Hindi requires 2.83 tokens per word versus 1.15 for English—a $7.99\times$ attention cost penalty. KV cache simulation across five hardware tiers (H100 to RTX 4060) reveals a $21\times$ gap: consumer GPUs OOM at 26K tokens while frontier GPUs sustain 557K. In multi-turn serving with 32 concurrent users, Hindi OOMs at turn 2 on RTX 4060 versus turn 5 for English. Prefill-decode analysis shows decode dominates 85.6–99.6% of latency. Our Composite Resource Divide Index (CRDI) reveals $8.1\times$ structural disparity between Swahili (0.807) and English (0.100).

I. Introduction

The rapid proliferation of large language models has centralized AI capabilities despite open-weight releases [1]. The United States controls approximately 74% of global high-end AI compute, with 160 nations classified as “compute deserts” entirely dependent on foreign infrastructure [2][3]. This concentration is institutionalized through the CHIPS Act’s \$52.7 billion appropriation with explicit prohibitions on semiconductor expansion in China [4], and export controls on H100/H200 GPUs create structural access asymmetries.

The AI divide extends to data. English dominates 49.7% of global web content, while Swahili—spoken by 200 million people—constitutes merely 0.0025% [5]. Hindi, with 600 million speakers, accounts for only 3.77% of internet content. Of approximately 7,150 world languages, only 42 enjoy meaningful LLM support [6][7]. This data imbalance creates a self-reinforcing cycle: less digital content leads to poorer language models, which reduces adoption, which discourages content creation.

This paper investigates how compute and data disparities manifest as compounding performance gaps. We quantify the “tokenization tax,” the “memory wall,” and their interaction in serving scenarios. Our unified configuration framework ensures cross-experiment numerical consistency.

II. Related Work

GovAI identifies compute as uniquely excludable, making it the primary lever for AI regulation [1]. Nations without territorial compute clusters lack “computational sovereignty” [2]. Lundin et al. demonstrate that each additional token per word reduces downstream accuracy by 8–18 percentage points, coining the “tokenization tax” [8]. Hooper et al. identify KV cache as the fundamental bottleneck for long-context inference [9]. Recent work establishes that prefill (compute-bound) and decode (memory-bound) phases have distinct hardware requirements [10][11].

III. Methodology and Experimental Framework

All experiments employ a unified configuration system (`config.py`) ensuring numerical consistency across simulations. We model Llama-3-8B with 4-bit AWQ quantization (4.01 GB weights, standard for production serving) across five hardware tiers.

A. Unified Configuration Architecture

Our configuration framework centralizes all experimental parameters:

Model Specifications: Llama-3-8B with 8.03B parameters, 32 layers, 8 KV heads, 128 head dimension. KV cache size: 131,072 bytes/token (0.125 MB) in FP16.

GPU Hardware: All specifications sourced from NVIDIA official datasheets:

GPU	VRAM	Bandwidth	Compute (FP16)	Cost/hr	Tier
H100 SXM5	80GB	3.35 TB/s	495 TFLOPS	\$4.00	Frontier
A100 80GB	80GB	2.04 TB/s	312 TFLOPS	\$2.50	High-end
RTX 3090	24GB	936 GB/s	71 TFLOPS	\$0.80	Workstation
T4	16GB	320 GB/s	65 TFLOPS	\$0.80	Commodity
RTX 4060	8GB	272 GB/s	15 TFLOPS	\$0.30	Consumer

Table I: GPU hardware specifications (sources: NVIDIA datasheets).

Language Ecosystem Data: We integrate six metrics from authoritative sources: speaker populations (Ethnologue 2024), internet content percentage (W3Techs 2024), digital literacy proxy (ITU 2024), LLM MMLU accuracy (multilingual benchmarks), tokenization fertility (our measurements), and Wikipedia contributors (Wikimedia 2024).

B. Tokenization Fertility Methodology

We measure fertility using the real `meta-llama/Llama-3.2-1B` tokenizer via HuggingFace Transformers. Test texts include news articles, conversational dialogue, and technical content for each language. For non-Latin scripts (Arabic, Devanagari, Han), we verify word boundaries against native speaker references.

Language	Script	Fertility	Attn. Cost
Chinese	Han	0.73	0.53
English	Latin	1.15	1.32
Spanish	Latin	1.68	2.81
Swahili	Latin	2.34	5.47
Arabic	Arabic	2.40	5.76
Hindi	Devanagari	2.83	7.99

Table II: Tokenization fertility and attention cost multiplier (fertility²).

Hindi requires $2.46\times$ more tokens per word than English. Because attention scales as $O(n^2)$, this produces a $6\times$ attention cost penalty.

C. Memory Wall Simulation

KV cache grows linearly with sequence length. Available memory after model weights (4.01 GB) and 10% activation overhead:

Hardware	Available KV	Max Seq (B=1)	Max Seq (B=32)
H100	67.98 GB	556,933 tok	17,404 tok
A100	67.98 GB	556,933 tok	17,404 tok
RTX 3090	17.59 GB	144,056 tok	4,502 tok
T4	10.38 GB	85,073 tok	2,658 tok
RTX 4060	3.19 GB	26,091 tok	815 tok

Table III: Maximum sequence length before OOM (Llama-3-8B 4-bit).

The consumer RTX 4060 hits the memory wall at just 815 tokens for batch size 32—less than one page of text. The H100 sustains $21\times$ more concurrent tokens.

D. End-to-End Latency and Resource Divide Index

Latency comprises prefill (time-to-first-token, TTFT) and decode phases. For an identical 100-word task:

Language	GPU	Prefill (ms)	Decode (ms)	Total (ms)
English	H100	4.4	98.0	102.4
English	T4	33.2	1026.1	1059.3
Hindi	H100	10.9	247.7	258.6
Hindi	T4	82.6	2593.5	2676.1

Table IV: End-to-end latency for 100-word task, 50-word output.

English on H100: 102.4 ms. Hindi on T4: 2,676.1 ms—a $26\times$ gap. We define the Resource Divide Index: $RDI = \text{fertility}^2 / (\text{bandwidth} \times \text{VRAM})$. Comparing extremes (Hindi on T4 vs English on H100), the RDI ratio reaches $1,010\times$.

E. Multi-Turn Conversational Serving

Real-world LLM applications operate as multi-turn conversations. We simulate 32 concurrent users with 80 words per turn:

Hardware	English	Hindi	Swahili	Chinese	Spanish	Arabic
H100	20+	20+	20+	20+	20+	20+
A100	20+	20+	20+	20+	20+	20+
T4	15	6	6	20+	10	7
RTX 4060	5	2	2	20+	4	2

Table V: Maximum turns before OOM (batch=32, 80 words/turn).

The RTX 4060 OOMs after 2 turns for Hindi versus 5 for English— $2.5\times$ difference in service capacity.

F. Prefill-Decode Disaggregation

LLM inference has two distinct phases: prefill (compute-bound, 85% GPU utilization) and decode (memory-bound, 20–40% utilization). Table VI shows decode dominates 85.6–99.6% of latency.

Workload	Prefill (ms)	Decode (ms)	Decode %
Short Q&A	1.9	51.5	96.4%
Medium Essay	19.2	522.0	96.4%

Workload	Prefill (ms)	Decode (ms)	Decode %
Long Document	162.6	967.9	85.6%
Code Generation	7.7	1378.7	99.4%

Table VI: Prefill vs decode breakdown (H100, Llama-3-8B 4-bit).

Assigning prefill to H100 and decode to RTX 4060 halves serving cost (\$0.00185 vs \$0.00373 per request) but increases latency by $13\times$ (7,059 ms vs 541 ms).

G. Language Ecosystem Analysis

We synthesize six metrics into a Composite Resource Divide Index (CRDI) with explicit weights: internet content (25%), LLM accuracy (20%), speakers (10%), digital literacy (15%), fertility (15%), Wikipedia (15%).

Language	Speakers	Internet %	CRDI	Tier
Swahili	200M	0.0025%	0.807	Low
Hindi	600M	3.77%	0.750	Moderate
Bengali	270M	0.20%	0.739	Low
Arabic	420M	3.65%	0.687	Moderate
English	1500M	49.7%	0.100	High

Table VII: Language ecosystem statistics and CRDI.

Swahili scores worst (0.807) despite 200 million speakers, due to minuscule digital content (0.0025%) and low LLM accuracy (45%). English enjoys $8.1\times$ structural advantage.

IV. Dual Viewpoint Analysis

Viewpoint A (Bo Pang): Structural Governance Imperative

The compute gap is not self-correcting: a T4’s GDDR6 cannot upgrade to HBM3e, and export controls prevent frontier hardware acquisition [1][4]. The tokenization tax is structural—Hindi’s 2.83 tokens per word stems from English-centric BPE training on 15 trillion tokens where Hindi is negligible [8]. Market incentives do not fund inclusive tokenizer redesign. Policy recommendations: (1) Sovereign AI funds for regional compute clusters; (2) International Compute Commons; (3) Mandatory inclusive tokenizer design for large models.

Viewpoint B (Connor Sempf): Data-Driven Self-Correction

While Viewpoint A identifies compute barriers, data is the root cause. CRDI shows LLM accuracy correlates strongly with internet content ($r=0.66$) and negatively with fertility ($r=-0.67$). Without representative corpora, no compute bridges the gap. Targeted investment in data commons breaks the cyclic disadvantage—10 billion Swahili tokens cost orders of magnitude less than H100 clusters, and data is non-excludable once released.

Synthesis

Both viewpoints identify real barriers. English benefits across all dimensions—no single intervention can replicate this. Compounding disadvantage implies compounding returns from coordinated action: inclusive tokenizers reduce fertility, data commons improve model quality, compute sharing enables deployment. Neither governance nor data investment alone suffices.

V. Conclusion

The AI divide is real, quantifiable, and compounding: (1) Hindi pays $7.99\times$ attention cost versus English’s $1.32\times$; (2) Consumer GPUs OOM at 26K tokens while frontier GPUs sustain 557K; (3) Hindi OOMs $2.5\times$ faster than English in serving; (4) Decode dominates 85.6–99.6% of latency; (5) CRDI reveals $8.1\times$ structural disparity. Moving forward requires international data commons, regional compute-sharing with CRDI-based metrics, and mandatory inclusive tokenizer design.

References

[1] GovAI, “Computing Power and the Governance of AI,” arXiv:2409.02888, 2024. [2] V. Lehdonvirta et al., “Computational Sovereignty and the AI Divide,” Proc. AIES, 2024. [3] Tony Blair Institute, “State of Compute Access,” Policy Report, 2024. [4] U.S. Congress, “CHIPS and Science Act of 2022,” Public Law 117-167, 2022. [5] W3Techs, “English accounts for 49.40% of internet content,” 2024. [6] M. Abbott et al., “Masakhane—Machine Translation for Africa,” arXiv:2003.11529, 2020. [7] A. Adebara et al., “The State of LLMs for African Languages,” arXiv:2506.02280, 2025. [8] J. Lundin et al., “The Token Tax: Systematic Bias in Multilingual Tokenization,” arXiv:2509.05486, 2025. [9] C. Hooper et al., “KVQuant: Towards 10 Million Context Length,” UC Berkeley EECS, 2024. [10] WVA, “A Global Optimization Control Plane for LLM Inference,” arXiv:2603.09730, 2026. [11] DUET, “Disaggregated Hybrid Mamba-Transformer LLMs,” 2025.

Appendix: Compute and AI Usage Disclosure

Compute: Georgia Tech AI Makerspace, ~12 hours across 3 sessions. All experiments are CPU-runnable simulations. RTX 3090 used for tokenizer validation only. Total: ~20 hours.

AI Usage: Per Georgia Tech policy: code developed with AI assistance for boilerplate and matplotlib styling; literature research AI-assisted but manually verified; writing AI-assisted for organization; CRDI, RDI metrics, and simulation frameworks are original contributions.

Code Repository: https://github.com/Pb314314/Fund_ML